

Towards QoS Prediction based on Temporal Transformers for IoT Applications

Aroosa Hameed, John Violos, Aris Leivadeas *Senior*

Member, IEEE, Nina Santi, Rémy Grünblatt, and Nathalie Mitton *Member, IEEE*

Abstract

Internet of Things (IoT) devices generate a tremendous amount of time series data that is extremely dynamic, heterogeneous and time dependent. Such types of data introduce significant challenges for the real-time prediction of QoS metrics of IoT applications with different traffic characteristics. To this end, in this paper, we propose a temporal transformer model and a unified system to predict several QoS metrics of heterogeneous IoT applications when they communicate with the Edge of the network. The transformer model also leverages an attention module to provide a solution for both short-term and long-term sequence prediction of QoS metrics that allows to better extract any time dependencies. In particular, in our framework, we firstly generate a set of datasets containing real-time traffic information of five different IoT applications such as Heating, Ventilation, and Air Conditioning (HVAC), lighting, Voice over Internet Protocol (VoIP), surveillance and emergency response using the 802.15.4 access technology and the RPL routing protocol. Following, we perform the data cleaning, downsampling and pre-processing of the datasets and we construct the QoS datasets, which include four QoS metrics, namely throughput, packet delivery ratio, packet loss ratio and latency. Finally, we evaluate the transformer model through extensive experimentation using both short-term and long-term dependencies and we show that our model can guarantee a robust performance and accurate QoS prediction.

Index Terms

Deep learning, Edge computing, Internet of Things, QoS prediction, time series, transformer

I. INTRODUCTION

The number of Internet of Things (IoT) applications have considerably increased, while generating a tremendous amount of data. According to Cisco, the number of connected devices will reach up to 14.7 billion by 2023 [1]. The devices are expected to continuously generate large volumes of data requiring extensive analysis to capture valuable information that can help in the intelligent decision making. However, the device's CPU, memory, and disk capacity restricts the data processing on the device itself. Thus, the data and the analysis processing have to be offloaded to more resource powerful platforms, such as the newly introduced Edge Computing [2]. Edge computing can facilitate the data processing very close to the source of the data, reducing thus the overall latency perceived. In this way, the processing burden is shifted to the Edge of the network through a process called task offloading [3].

IoT applications consist of heterogeneous devices that send data of different contexts, with different reporting frequencies usually over a random access channel generating thus, high

A. Hameed, J. Violos and A. Leivadeas are with the Department of Software and Information Technology Engineering, École de Technologie Supérieure, University of Quebec, Montreal, Canada. N. Santi and N. Mitton are with INRIA Lille-Nord, France. R. Grünblatt is with SAMOVAR, Télécom SudParis, Institut Polytechnique de Paris, Évry, France. E-mails: {aroosa.hammed, ioannis.violos}.1@ens.etsmtl.ca, aris.leivadeas@etsmtl.ca, {nina.santi, nathalie.mitton}.inria.fr, remy@grunblatt.org

interference levels [4]. All these add several levels of complexity when it comes to the prediction of typical Key Performance Indicators (KPIs) in IoT. Hence, it is necessary to propose an efficient model that will analyze and predict the QoS metrics using IoT time series data.

To overcome the research gaps in current studies, the main contributions of this paper are:

- We consider 5 different IoT smart building applications that present different requirements in terms of number of devices, packet length, context of message, and message frequency transmission. We deploy the applications in a real testbed [5] comprised of approximately 300 IoT devices and generate data over an IEEE 802.15.4 access network.
- We provide predictions of four major QoS metrics: Throughput, Packet Delivery Ratio (PDR), Packet Loss Ratio (PLR) and Latency. We provide both univariate and multivariate multi-step prediction for all four QoS metrics of the five IoT applications.
- We design and implement a QoS prediction mechanism based on Temporal Transformers that models temporal dependencies within input sequences consisting of IoT data, and that is able to handle the long input sequences with the attention module to make prediction.

II. RELATED WORK

A. Deep Learning for QoS Prediction

Abdellah et al. [6] predicted the delay using a nonlinear autoregressive exogenous (NARX) RNN following both a single-step and a multi-step ahead prediction. The authors in [7] predicted delay using a simple Deep Neural Network (DNN). Said and Tolba [8] proposed a deep learning model that predicts throughput, delay, and packet loss of an IoT communication system. Hou et al. [9] attempted to predict the throughput using a Convolutional Neural Network (CNN) with target vectorization.

Fan et al. [10] proposed a deep learning based RNN model using an attention mechanism for IoT data processing at the Edge, called UrbanEdge. Wu et al. [11] proposed EdgeLSTM, an Edge-based deep learning system utilizing grid LSTM along with Support Vector Machine (SVM). Abdellah et al. [12] performed prediction of throughput of IoT traffic in a 5G network using LSTM. Lopez-Martin et al. [13] proposed forecasting of IoT traffic using a stochastic gradient descent algorithm and a neural network architecture called gaNET.

B. Deep Learning for Time Series Forecasting

Hochreiter and Schmidhuber [14] introduced LSTM that uses a gating mechanism for controlling access to memory cell and mitigates the vanishing gradient problem. Sutskever et al. [15] proposed stacked LSTM for sequence-to-sequence learning. Cheng et al. [16] proposed bidirectional LSTM. Bai et al. [17] proposed a Temporal Convolutional Network (TCN) combining dilations and residual connections. Salinas et al. [18] proposed DeepAR for probabilistic forecasting using autoregressive recurrent networks. Lai et al. [19] proposed LSTNet, combining convolutional and recurrent layers for multivariate time series forecasting.

C. Limitations of Related Work

The limitations of the above mentioned works can be summarized as: (i) Most studies provide prediction of IoT traffic type and do not predict QoS attributes [10]–[13]; (ii) Some existing studies provide prediction as a univariate forecast [9]–[11]; (iii) Existing studies based on neural networks are mostly designed for short-term sequence prediction, and RNN based models have the vanishing gradient problem which prevents training over long sequences of data [14], [15], [18].

III. PROBLEM FORMULATION

The IoT applications are represented by the set $\mathcal{A} = \{a_1, a_2, a_3, a_4, a_5\}$ and the set $\mathcal{D} = \{d_{a_1}^1, d_{a_2}^2, \dots, d_{a_i}^m\}$ represents the data generated by each application. Each network dataset is constituted by sending and receiving information:

$$D = \{(u_{a_1}^1, s_{a_1}^1), (u_{a_2}^2, s_{a_2}^2), (u_{a_3}^3, s_{a_3}^3), (u_{a_4}^4, s_{a_4}^4), (u_{a_5}^5, s_{a_5}^5)\}$$

The four QoS metrics are represented as:

$$Q = \{q_1^1, q_2^2, \dots, q_i^t\} \quad (\text{Throughput}) \quad (1)$$

$$P = \{p_1^1, p_2^2, \dots, p_i^t\} \quad (\text{PDR}) \quad (2)$$

$$E = \{e_1^1, e_2^2, \dots, e_i^t\} \quad (\text{PLR}) \quad (3)$$

$$L = \{l_1^1, l_2^2, \dots, l_i^t\} \quad (\text{Latency}) \quad (4)$$

In the Time Series Forecasting (TSF) setting, let $X = \{x^1, x^2, \dots, x^N\}^T$ represent the multi-variate QoS time series with N variables, T as timestamp and $X \in \mathbb{R}^{T \times N}$. Given the input time sequence $\{x_1^t, x_2^{t+1}, \dots, x_\tau^{t+\tau}\} \subset X$, the goal is to learn a precise forecasting model:

$$\mathcal{M} : X_{t,\tau}^{(i)} \rightarrow \tilde{X}_{t,h}^{(i+\tau)}$$

by minimizing a loss function, where h is the forecast horizon.

IV. DATASET CONSTRUCTION

A. Applications and Edge Computing Infrastructure

Five different IoT applications are considered: (1) **Emergency Response**, (2) **HVAC**, (3) **Surveillance**, (4) **VoIP**, and (5) **Lighting**. Table I summarizes the experimentation parameters for each application. Each experiment involves three types of nodes: SERVER, BORDER ROUTERS, and SENSORS, connected via the 802.15.4 access protocol and IPv6 RPL routing [20].

TABLE I: Experimentation Parameters

Scenario	Sensors	Routers	Duration (s)	Pkt Len (B)	Gen. Type	Lambda / Period
Surveillance	10	3	10090	127	Exponential	$\lambda = 196.74$
Emergency Resp.	40	5	10090	127	Hybrid	$\lambda = 0.0333, T = 30s$
HVAC	100	5	10090	60	Periodic	$T = 260s$
Lighting	100	5	10090	30	Exponential	$\lambda = 0.00208$
VoIP	10	1	10090	127	Hybrid	$\lambda = 15.74, T = 0.063s$

B. Feature Engineering and QoS Metrics

Raw features include transmitting node name, timestamp, payload size, destination address, destination port, and hexadecimal payload. From these, we engineer the following QoS metrics for each 5 ms time interval t :

Packet Delivery Ratio (PDR):

$$PDR = \frac{\text{totalrec_pack}}{\text{totaltrans_pack}} \times 100 \quad (5)$$

Packet Loss Ratio (PLR):

$$PLR = \frac{\text{totalloss_pack}}{\text{totalrec_pack}} \times 100 \quad (6)$$

Throughput:

$$\text{Throughput} = \frac{\text{totalrec_pack}}{\text{timelast_pack} - \text{timefirst_pack}} \quad (7)$$

Transmission Latency:

$$\text{Latency} = \frac{\sum_{t_pack} (\text{timerec_pack} - \text{timetrans_pack})}{\text{totaltrans_pack}} \quad (8)$$

C. Data Preprocessing

After feature engineering, outliers are removed and missing values are filled by average values. Finally, min-max normalization is applied:

$$X_{\text{normalized}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (9)$$

where x is the original QoS value, $x_{\min}$ and $x_{\max}$ are the minimum and maximum values of that feature, respectively. The normalized data lie in the range $[0, 1]$.

Statistical characteristics of all QoS datasets are summarized in Table II.

TABLE II: Statistical Characteristics of QoS Datasets for All IoT Applications

App	Throughput			PDR			PLR			Latency		
	Mean	S.D.	Med.	Mean	S.D.	Med.	Mean	S.D.	Med.	Mean	S.D.	Med.
HVAC	0.254	0.205	0.191	0.331	0.391	0.000	0.233	0.238	0.125	0.045	0.059	0.029
VoIP	0.324	0.109	0.304	0.533	0.207	0.548	0.501	0.060	0.495	0.005	0.002	0.005
Lighting	0.165	0.185	0.095	0.100	0.226	0.000	0.038	0.101	0.000	0.042	0.075	0.012
Emergency	0.062	0.062	0.043	0.258	0.227	0.174	0.134	0.127	0.085	0.066	0.073	0.031
Surveillance	0.337	0.168	0.330	0.291	0.126	0.295	0.080	0.129	0.035	0.380	4.871	0.000

V. PROPOSED TEMPORAL TRANSFORMER FRAMEWORK*A. Overview*

Figure 1 illustrates the proposed system. Raw IoT data are generated for five applications, features are engineered (Section IV), and the preprocessed QoS datasets are divided into training (first 5 days), validation (day 6), and testing (day 7) sets. The temporal transformer model is then trained and evaluated.

B. Temporal Transformer Architecture

The model is based on the transformer encoder architecture [21], [22]. The decoder is omitted since the prediction horizon is predefined. The temporal transformer consists of three modules: *Input Embedding*, *Encoder Module*, and *Output Module*.

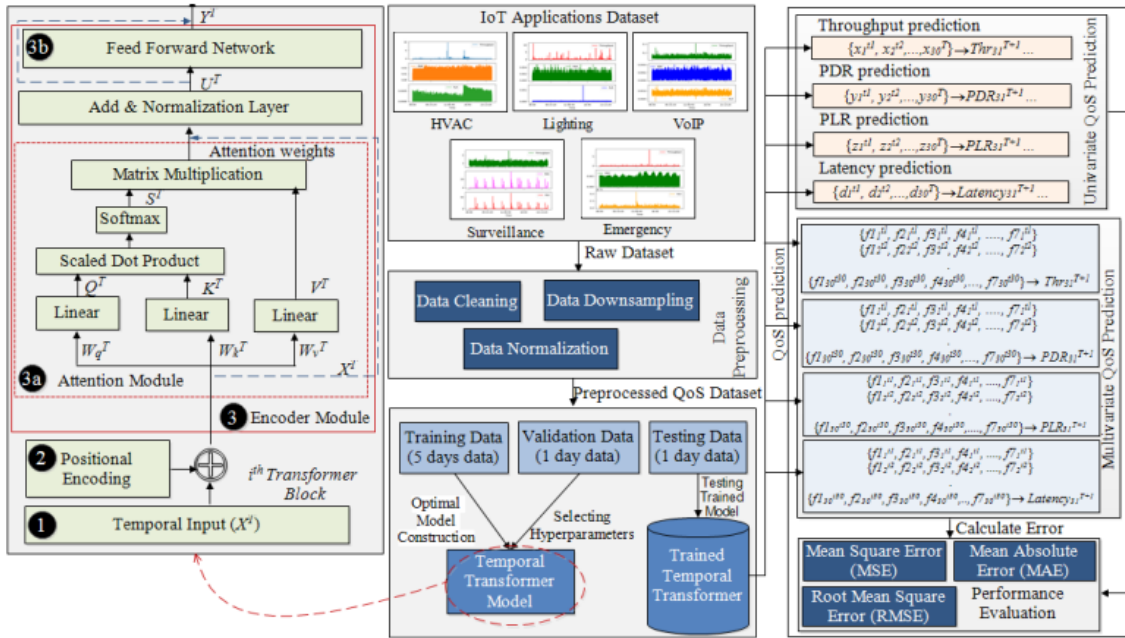


Fig. 1: Overview of proposed system for QoS Prediction.

1) *Input and Positional Encoding*: A rolling window strategy with window size 30 is applied. Sinusoidal positional encoding is used to inject positional information:

$$PE_{pos,2i} = \sin\left(\frac{pos}{10000^{2i/d_{mod}}}\right), \quad 0 < pos < N - 1 \quad (10)$$

$$PE_{pos,2i+1} = \cos\left(\frac{pos}{10000^{(2i+1)/d_{mod}}}\right), \quad 0 < pos < N - 1 \quad (11)$$

where PE denotes the positional encoding, pos is the position index, $2i$ represents even dimensions and $2i + 1$ the odd dimensions of d_{mod} .

2) *Multi-Head Attention*: The core of the transformer is the Multi-Head Attention (MHA) mechanism based on scaled dot-product:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (12)$$

where Q (query), K (key), and V (value) represent QoS values within the input sequence at specific time steps, and d_k is the dimension of the key.

The multi-head attention is computed as:

$$h_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \quad (13)$$

$$\text{MultiHead}(Q, K, V) = \text{concat}(h_1, h_2, \dots, h_n)W^0 \quad (14)$$

where W_i^Q , W_i^K , W_i^V are linear transformations for the i -th head, n is the total number of heads, and W^0 is the linear transformation of the concatenated output.

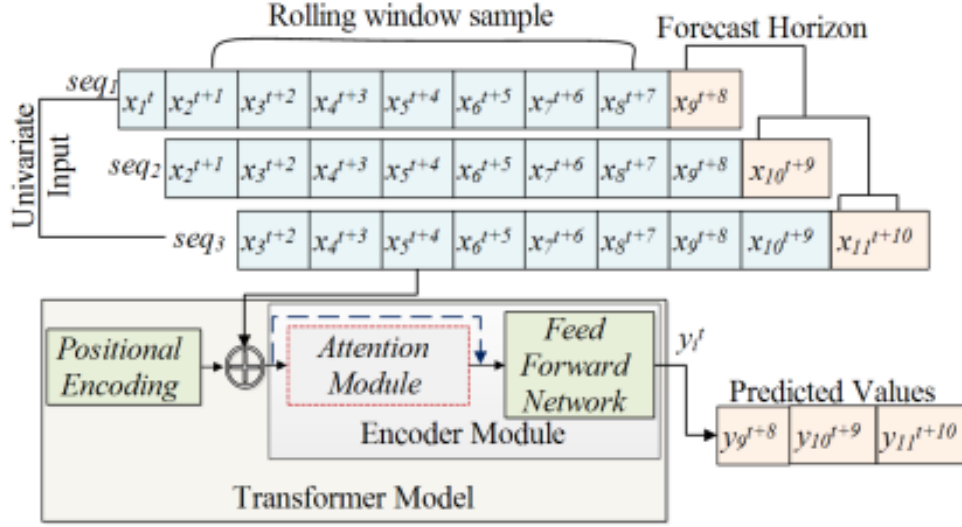


Fig. 2: Overview of univariate input and output of prediction.

3) *Feed Forward Network*: The Feed Forward Network (FFN) consists of linear transformations and a Conv1D layer with ReLU activation:

$$FFN(x) = \text{ReLU}(0, xW_1 + b_1)W_2 + b_2 \quad (15)$$

where W_1, W_2 are weights, b_1, b_2 are biases, and x is the normalized output of the MHA module.

C. Algorithm Description

Algorithm 1 presents the full QoS prediction procedure, and Algorithm 2 details the temporal transformer construction.

Algorithm 1 QoS Prediction Algorithm

Require: QoS training D_{train} , validation D_{val} , testing D_{test}

- 1: $m \leftarrow \text{BUILDMODEL}(D_{\text{train}}, D_{\text{val}})$
- 2: $\text{tuner} \leftarrow \text{RANDOMSEARCH}(m, \text{obj}, \text{maxtr}, \text{searchtr}) \quad \triangleright \text{obj: maximize validation accuracy}$
- 3: $\text{model} \leftarrow \text{BESTMODEL}(\text{tuner}, \text{numm})$
- 4: $\text{history} \leftarrow \text{model.fit}(D_{\text{train}}, D_{\text{val}}, \text{epochs})$
- 5: $\tilde{Y} \leftarrow \text{model.predict}(D_{\text{test}})$

Ensure: Future QoS values $\tilde{Y} = \{y_1^t, y_2^{t+1}, \dots, y_{h-1}^{T-1}, y_h^T\}$

D. Complexity Analysis

Proposition 1: The computational complexity of Algorithm 1 is $O(n^2d)$.

Proposition 2: The computational complexity of the INPUT EMBEDDING module is $O(nd)$.

Proposition 3: The computational complexity of the ENCODER MODULE is $O(n^2d)$, dominated by the self-attention dot-product operation ($n \times d$ matrix multiplied by $d \times n$).

Algorithm 2 Temporal Transformer Algorithm

INPUT EMBEDDING($D_{\text{train}}, pos, dim$)

- 1: $input \leftarrow \text{INPUTLAYER}(D_{\text{train}})$
- 2: $pos \leftarrow \text{POSITIONALENCODING}(pos, dim)$
- 3: $embres \leftarrow \text{ADD}(input, pos)$ ▷ Module 1

ENCODER MODULE($embres, hs, numh, drate, fil, ks, act$)

- 4: $x \leftarrow \text{LAYERNORM}(embres)$
- 5: $x \leftarrow \text{MHA}(hs, numh, drate, x)$
- 6: $x \leftarrow \text{DROPOUT}(x)$
- 7: $res \leftarrow x + input$ ▷ MHA
- 8: $x \leftarrow \text{LAYERNORM}(res)$
- 9: $x \leftarrow \text{CONV1D}(fil, ks, act, x)$
- 10: $x \leftarrow \text{DROPOUT}(drate, x)$ ▷ Module 2

OUTPUT MODULE(x, res)

- 11: $x \leftarrow \text{GLOBALAVGPOOLING1D}(x)$
- 12: $x \leftarrow \text{DENSE}(x)$
- 13: $x \leftarrow \text{ADD}(x, res)$
- 14: $x \leftarrow \text{LAYERNORM}(x)$ ▷ Module 3

The overall complexity of the proposed temporal transformer (Algorithm 2) is:

$$O(nd) + O(n^2d) + O(n) = O(n^2d)$$

VI. PERFORMANCE EVALUATION

A. Evaluation Setup and Metrics

Each dataset is zero-mean normalized and standardized. The window size is set to 30. All models were trained on NVIDIA V100 (16 GB) and NVIDIA P100 (16 GB) GPUs from Compute Canada. Three metrics are used:

$$MAE = \frac{1}{n} \sum_{j=1}^n |y_j - \hat{y}_j| \quad (16)$$

$$MSE = \frac{1}{n} \sum_{j=1}^n (y_j - \hat{y}_j)^2 \quad (17)$$

$$RMSE = \sqrt{\frac{1}{n} \sum_{j=1}^n (y_j - \hat{y}_j)^2} \quad (18)$$

B. Baselines and Hyperparameters

Baselines for univariate prediction: MLP, Stacked LSTM, Bidirectional LSTM. Baselines for multivariate prediction: Stacked LSTM, Bidirectional LSTM, and LSTNet [19]. Hyperparameters were tuned using Keras random search with 5 trials and 100 epochs. Table III summarizes the selected hyperparameters for univariate throughput prediction.

TABLE III: Hyperparameters for Univariate Throughput Prediction Across All Datasets

Model	Hyperparameter	Min	Max	HVAC	VoIP	Lighting	Emergency	Surveillance
MLP	Neurons	8	512	64	392	288	40	504
	Dropout	0	0.5	0.1	0.3	0.2	0.1	0.1
	Learning rate	1e-4	1e-2	0.01	0.0001	0.001	0.001	0.0001
Stacked LSTM	Neurons	8	128	24	72	104	96	16
	Dropout	0	0.5	0.4	0.001	0.1	0.4	0.4
	Layers	2	6	3	2	4	3	5
Bidir. LSTM	Neurons	8	512	32	16	24	64	352
	Dropout	0	0.5	0.5	0.4	0.1	0.1	0.2
Temporal Trans.	Head size	4	256	28	32	128	4	2
	Num. heads	4	32	6	18	24	4	3
	Dropout	0	0.5	0.2	0.5	0.2	0.5	0.2
	Trans. blocks	4	16	16	8	4	4	2
	Linear neurons	4	128	96	84	52	64	116
	Filters	4	64	96	28	52	64	8
	Attn. layers	1	15	4	5	2	2	3

C. Univariate Results

1) *Throughput*: Table IV presents the univariate throughput results. The temporal transformer provides the least error for almost all datasets. For the emergency application, the transformer reduces MAE by 89% over the second-best method. Figures 3a and 3b visualize the MSE and MAE comparisons.

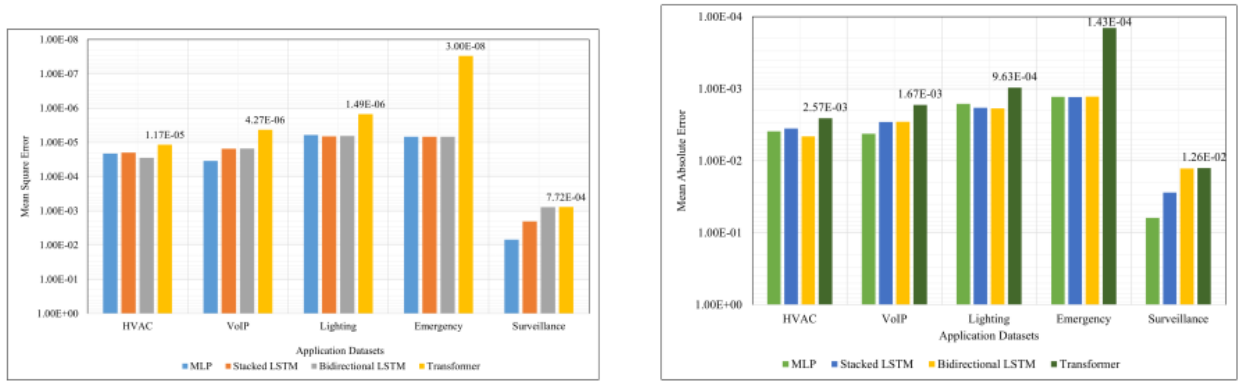
TABLE IV: Univariate Forecasting Results for Throughput (best results in **bold**)

Dataset	MLP			Stacked LSTM			Bidir. LSTM			Temp. Transformer		
	MAE	MSE	RMSE	MAE	MSE	RMSE	MAE	MSE	RMSE	MAE	MSE	RMSE
HVAC	3.91e-3	2.11e-5	4.60e-3	3.56e-3	2.0e-5	4.48e-3	4.57e-3	2.82e-5	5.31e-3	2.57e-3	1.17e-5	3.42e-3
VoIP	4.20e-3	3.49e-5	5.91e-3	2.89e-3	1.53e-5	3.92e-3	2.87e-3	1.52e-5	3.9e-3	1.67e-3	4.27e-6	2.07e-3
Lighting	1.62e-3	6.13e-6	2.47e-3	1.83e-3	6.67e-6	2.58e-3	1.87e-3	6.52e-6	2.55e-3	9.63e-4	1.49e-6	1.22e-3
Emergency	1.29e-3	6.86e-6	2.62e-3	1.3e-3	6.88e-6	2.62e-3	1.28e-3	6.87e-6	2.62e-3	1.43e-4	3.0e-8	1.73e-4
Surveillance	6.22e-2	6.92e-3	8.32e-2	2.76e-2	2.03e-3	4.5e-2	1.28e-2	7.74e-4	2.78e-2	1.26e-2	7.72e-4	2.78e-2

2) *PDR, PLR, and Latency*: Tables V, VI, and VII present the results for PDR, PLR, and latency, respectively. Across these metrics, the temporal transformer achieves the best performance on 18/20 settings for MAE, 19/20 for MSE, and 19/20 for RMSE. For latency, the transformer improves MAE by 85% for HVAC, 47% for lighting, and 41% for surveillance over the second-best baseline.

D. Multivariate Results

Tables VIII and IX present the multivariate throughput and latency results. Figure 4 shows the MAE comparisons. The proposed transformer achieves 20/20 best MAE settings for the multivariate task. For the surveillance application, the transformer provides a 32% improvement



(a) MSE of univariate throughput prediction across all datasets.

(b) MAE of univariate throughput prediction across all datasets.

Fig. 3: Univariate throughput prediction error comparison.

TABLE V: Univariate Forecasting Results for PDR (best results in **bold**)

Dataset	MLP			Stacked LSTM			Bidir. LSTM			Temp. Transformer		
	MAE	MSE	RMSE	MAE	MSE	RMSE	MAE	MSE	RMSE	MAE	MSE	RMSE
HVAC	4.41e-3	2.60e-5	5.10e-3	4.40e-3	2.65e-5	5.15e-3	4.35e-3	2.54e-5	5.04e-3	4.15e-3	2.73e-5	5.23e-3
VoIP	2.71e-5	1.0e-9	3.16e-5	2.31e-5	1.0e-9	2.93e-5	2.43e-5	9.47e-10	3.08e-5	2.31e-5	8.55e-10	2.93e-5
Lighting	2.47e-4	1.40e-7	3.74e-4	2.64e-4	1.44e-7	3.80e-4	2.65e-4	1.37e-7	3.70e-4	2.63e-4	1.37e-7	3.70e-4
Emergency	1.32e-4	3.0e-8	1.73e-4	1.32e-4	3.0e-8	1.73e-4	1.31e-4	2.9e-8	1.71e-4	8.06e-5	9.0e-9	9.73e-5
Surveillance	3.20e-5	2.0e-9	3.89e-5	2.05e-5	1.0e-9	2.65e-5	3.40e-5	1.96e-9	4.43e-5	2.0e-5	6.96e-10	2.53e-5

TABLE VI: Univariate Forecasting Results for PLR (best results in **bold**)

Dataset	MLP			Stacked LSTM			Bidir. LSTM			Temp. Transf	
	MAE	MSE	RMSE	MAE	MSE	RMSE	MAE	MSE	RMSE	MAE	MSE
HVAC	2.76e-5	1.22e-9	3.50e-5	2.36e-5	8.37e-10	2.89e-5	2.72e-5	1.16e-9	3.41e-5	2.27e-5	8.10e-10
VoIP	2.39e-5	1.0e-9	3.32e-5	2.61e-5	1.30e-9	3.60e-5	1.77e-5	1.0e-9	2.42e-5	1.70e-5	5.82e-10
Lighting	2.74e-6	3.74e-11	6.12e-6	3.80e-6	3.37e-11	5.81e-6	3.86e-6	3.32e-11	5.76e-6	3.75e-6	3.32e-11
Emergency	1.83e-12	5.40e-24	2.32e-12	2.16e-12	7.5e-24	2.74e-12	1.88e-12	5.3e-24	2.32e-12	1.80e-12	5.2e-24
Surveillance	2.33e-3	5.46e-6	2.34e-3	2.67e-5	1.65e-9	4.06e-5	4.76e-4	2.45e-7	4.96e-4	2.57e-5	1.39e-9

in throughput MAE compared to the best baseline, attributable to its superior handling of long-term fluctuating patterns via the attention mechanism. LSTNet consistently shows the worst performance across all applications, as it cannot handle the dynamic periodic and non-periodic patterns in the IoT datasets.

VII. CONCLUSION

In this work, we investigated the QoS prediction problem by formulating it as a univariate and multivariate time series forecasting problem. A new framework was introduced that promotes efficient QoS prediction for a number of coexisting and heterogeneous IoT applications. We

TABLE VII: Univariate Forecasting Results for Latency (best results in **bold**)

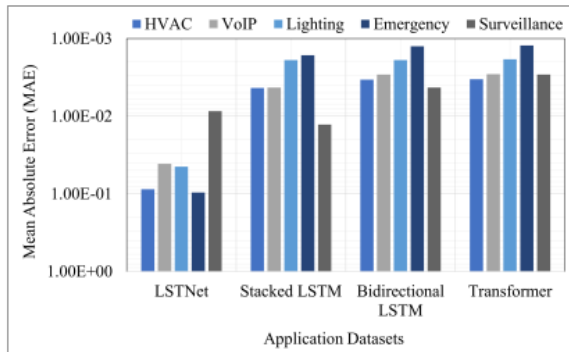
Dataset	MLP			Stacked LSTM			Bidir. LSTM			Temp. Transformer		
	MAE	MSE	RMSE	MAE	MSE	RMSE	MAE	MSE	RMSE	MAE	MSE	RMSE
HVAC	2.47e-2	9.07e-4	3.01e-2	2.60e-2	9.26e-4	3.04e-2	2.31e-2	7.27e-4	2.70e-2	3.36e-3	1.56e-5	3.95e-3
VoIP	9.69e-1	5.58e+2	2.36e+1	1.34e-3	2.75e-6	1.66e-3	1.42e-3	3.14e-6	1.77e-3	1.27e-3	2.28e-6	1.51e-3
Lighting	4.45e-2	4.09e-3	6.40e-2	4.44e-2	4.04e-3	6.36e-2	4.42e-2	4.04e-3	6.36e-2	2.34e-2	1.09e-3	3.30e-2
Emergency	4.63e-2	3.73e-3	6.10e-2	4.90e-2	4.07e-3	6.38e-2	4.84e-2	4.06e-3	6.37e-2	3.86e-2	2.67e-3	5.17e-2
Surveillance	2.74e-4	1.20e-7	3.46e-4	2.76e-4	2.35e-6	1.53e-3	2.63e-4	1.15e-7	3.39e-4	1.54e-4	3.16e-8	1.78e-4

TABLE VIII: Multivariate Forecasting Results for Throughput (best results in **bold**)

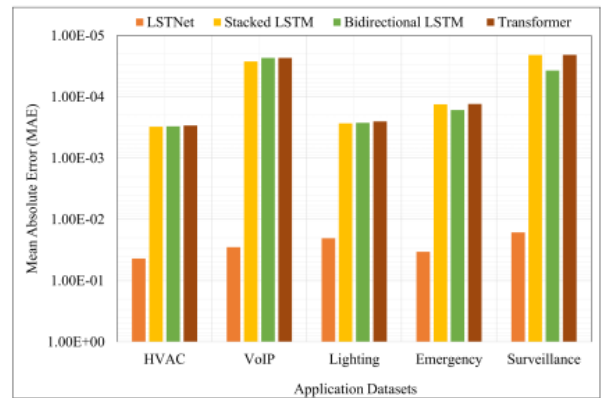
Dataset	LSTNet			Stacked LSTM			Bidir. LSTM			Temp. Transformer		
	MAE	MSE	RMSE	MAE	MSE	RMSE	MAE	MSE	RMSE	MAE	MSE	RMSE
HVAC	8.74e-2	7.63e-1	8.73e-1	4.34e-3	2.81e-5	5.30e-3	3.38e-3	1.17e-5	3.42e-3	3.35e-3	1.71e-5	4.14e-3
VoIP	4.10e-2	3.91e-1	6.25e-1	4.27e-3	3.32e-5	5.76e-3	2.91e-3	1.55e-5	3.94e-3	2.87e-3	1.48e-5	3.85e-3
Lighting	4.46e-2	6.88e-1	8.29e-1	1.90e-3	7.18e-6	2.68e-3	1.89e-3	7.17e-6	2.68e-3	1.85e-3	7.10e-6	2.66e-3
Emergency	9.65e-2	1.45e-1	3.81e-1	1.64e-3	7.71e-6	2.78e-3	1.26e-3	6.72e-6	2.59e-3	1.23e-3	6.67e-6	2.50e-3
Surveillance	8.60e-3	9.14e-2	3.02e-1	1.28e-2	7.73e-4	2.78e-2	4.29e-3	3.65e-5	6.04e-3	2.91e-3	1.55e-5	3.93e-3

TABLE IX: Multivariate Forecasting Results for Latency (best results in **bold**)

Dataset	LSTNet			Stacked LSTM			Bidir. LSTM			Temp. Transformer		
	MAE	MSE	RMSE	MAE	MSE	RMSE	MAE	MSE	RMSE	MAE	MSE	RMSE
HVAC	8.40e-2	2.81e0	1.66e0	2.56e-2	9.28e-4	3.05e-2	2.32e-2	7.81e-4	2.79e-2	2.27e-2	7.33e-4	2.71e-2
VoIP	2.41e-2	2.68e-1	5.05e-1	1.35e-3	2.85e-6	1.69e-3	1.35e-3	2.85e-6	1.69e-3	9.24e-4	1.18e-6	1.09e-3
Lighting	7.13e-2	1.52e0	1.23e0	3.55e-2	2.56e-3	5.06e-2	3.63e-2	2.56e-3	5.06e-2	2.27e-2	1.07e-3	3.27e-2
Emergency	4.18e-2	6.65e-1	8.10e-1	6.35e-2	6.75e-3	8.22e-2	4.77e-2	3.88e-3	6.23e-2	3.88e-2	2.26e-3	4.75e-2
Surveillance	2.41e-2	2.68e-1	5.05e-1	2.53e-4	1.16e-7	3.40e-4	2.49e-4	9.66e-8	3.11e-4	1.56e-4	3.26e-8	1.81e-4



(a) MAE of multivariate throughput prediction.



(b) MAE of multivariate PDR prediction.

Fig. 4: Multivariate prediction error comparisons across all datasets.

firstly generated five different real-time datasets for HVAC, lighting, VoIP, surveillance, and emergency response applications. Following, we presented a novel transformer-based architecture which learns temporal representations and their complex dependencies in a long-term fashion, for the prediction of four important QoS metrics, namely, throughput, PDR, PLR, and latency as defined in Equations (7)–(8). The transformer architecture leverages the attention mechanism (Equation (12)) and sinusoidal positional encoding (Equations (10)–(11)), which is effective at modelling time series. The extensive experimental evaluation proves that our proposed temporal transformer achieves superior performance for almost all of the five IoT applications and for both univariate and multivariate settings, as compared with several competitive time series baseline methods.

As future work, we aim to explore alternative attention techniques, such as sparse attention or compressed attention, and investigate their impact on accuracy. Furthermore, we would like to predict several key QoS metrics when mobile IoT devices are considered, thus creating another level of uncertainty in the overall communication.

REFERENCES

- [1] Cisco, “Cisco Annual Internet Report (2018–2023) White Paper,” <https://www.cisco.com/c/en/us/solutions/collateral/executive-perspectives/annual-internet-report/white-paper-c11-741490.html>, 2022, accessed: Feb. 2022.
- [2] A. Pratap, A. Kumar, and M. Kumar, “Analyzing the need of edge computing for internet of things (IoT),” in *Proc. Second Int. Conf. on Computing, Communications, and Cyber-Security*, Singapore, 2021, pp. 203–212.
- [3] F. Saeik *et al.*, “Task offloading in edge and cloud computing: A survey on mathematical, artificial intelligence and control theory solutions,” *Computer Networks*, vol. 195, p. 108177, 2021.
- [4] D. Dechouniotis *et al.*, “Edge computing resource allocation for dynamic networks: The DRUID-NET vision and perspective,” *Sensors*, vol. 20, no. 8, p. 2191, 2020.
- [5] C. Adjih *et al.*, “FIT IoT-lab: A large scale open experimental IoT testbed,” in *IEEE World Forum on IoT (WF-IoT)*, 2015, pp. 459–464.
- [6] A. R. Abdellah, O. A. Mahmood, and A. Koucheryavy, “Delay prediction in IoT using machine learning approach,” in *Int. Conf. on Ultra Modern Telecommunications and Control Systems and Workshops (ICUMT)*, 2020, pp. 275–279.
- [7] M. Ateeq, F. Ishmanov, M. K. Afzal, and M. Naeem, “Predicting delay in IoT using deep learning: A multiparametric approach,” *IEEE Access*, vol. 7, pp. 62 022–62 031, 2019.
- [8] O. Said and A. Tolba, “Accurate performance prediction of IoT communication systems for smart cities: An efficient deep learning based solution,” *Sustainable Cities and Society*, vol. 69, p. 102830, 2021.
- [9] Y. Hou *et al.*, “A study of throughput prediction using convolutional neural network over factory environment,” in *23rd Int. Conf. on Advanced Communication Technology (ICACT)*, 2021, pp. 429–434.
- [10] X. Fan, C. Xiang, L. Gong, X. He, C. Chen, and X. Huang, “UrbanEdge: deep learning empowered edge computing for urban IoT time series prediction,” in *ACM TURC ’19: Proc. of the ACM Turing Celebration Conference*, China, 2019, pp. 1–6.
- [11] D. Wu, H. Xu, Z. Jiang, W. Yu, X. Wei, and J. Lu, “EdgeLSTM: Towards deep and sequential edge computing for IoT applications,” *IEEE/ACM Transactions on Networking*, vol. 29, no. 4, pp. 1895–1908, 2021.
- [12] A. R. Abdellah, V. Artem, A. Muthanna, D. Gallyamov, and A. Koucheryavy, “Deep learning for IoT traffic prediction based on edge computing,” in *Distributed Computer and Communication Networks: Control, Computation, Communications*, 2020, pp. 18–29.
- [13] M. Lopez-Martin, B. Carro, and A. Sanchez-Esguevillas, “Neural network architecture based on gradient boosting for IoT traffic prediction,” *Future Generation Computer Systems*, vol. 100, pp. 656–673, 2019.
- [14] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [15] I. Sutskever, O. Vinyals, and Q. V. Le, “Sequence to sequence learning with neural networks,” in *Proc. NIPS*, 2014, pp. 3104–3112.
- [16] H. Cheng, Z. Xie, L. Wu, Z. Yu, and R. Li, “Data prediction model in wireless sensor networks based on bidirectional LSTM,” *EURASIP Journal on Wireless Communications and Networking*, vol. 2019, no. 1, 2019.
- [17] S. Bai, J. Z. Kolter, and V. Koltun, “An empirical evaluation of generic convolutional and recurrent networks for sequence modeling,” *arXiv preprint arXiv:1803.01271*, 2018.
- [18] D. Salinas, V. Flunkert, J. Gasthaus, and T. Januschowski, “DeepAR: Probabilistic forecasting with autoregressive recurrent networks,” *International Journal of Forecasting*, vol. 36, no. 3, pp. 1181–1191, 2020.
- [19] G. Lai, W. Chang, Y. Yang, and H. Liu, “Modeling long- and short-term temporal patterns with deep neural networks,” in *SIGIR ’18: The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval*, New York, NY, USA, 2018, pp. 95–104.

- [20] N. Santi *et al.*, “Automated and reproducible application traces generation for IoT applications,” in *Proc. of the 17th Symp. on QoS and Security for Wireless and Mobile Networks (Q2SWinet)*, 2021, pp. 17–24.
- [21] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, 2017, pp. 6000–6010.
- [22] G. Zerveas, S. Jayaraman, D. Patel, A. Bhamidipaty, and C. Eickhoff, “A transformer-based framework for multivariate time series representation learning,” in *KDD '21: Proc. of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, 2021, pp. 2114–2124.